Deep Networks with Stochastic Depth (Replication Study)

Jin Fan (jf936) · Yixuan Yang (yy2445) · Youlun Jiang (yj622)
GitHub: https://github.com/catttjyl/cs5482_finalproject_SD

1. Introduction

Very deep residual networks achieve strong expressiveness but suffer from vanishing gradients, diminishing feature reuse, and long training time. Ideally, we want to train a shorter network for efficient gradient flow, then use the full-depth network at test time. The paper *Deep Networks with Stochastic Depth* by Huang et al. addresses this dilemma by proposing a simple but effective training strategy: Stochastic Depth. The main idea is to randomly drop entire residual blocks during each mini-batch of training, bypassing them with identity connections. The method effectively reduces training cost and improves generalization through implicit regularization.

In this project, we re-implemented stochastic depth on CIFAR-10 using a 110-layer ResNet against a constant-depth ResNet baseline. Our goal was to reproduce the paper's central improvement in test error and training time and investigate why the method works through additional analyses of gradient behavior, feature representations, and robustness under label noise.

2. Chosen Result

We target the core accuracy and efficiency claim from Figure 1 of the original paper: that SD reduces the test error of a 110-layer ResNet on CIFAR-10 from ~6.41% (constant depth) to ~5.25%, while cutting training time by ~25% from Table 1. This result directly demonstrates the practical value of stochastic depth as both a regularizer and a training accelerator.

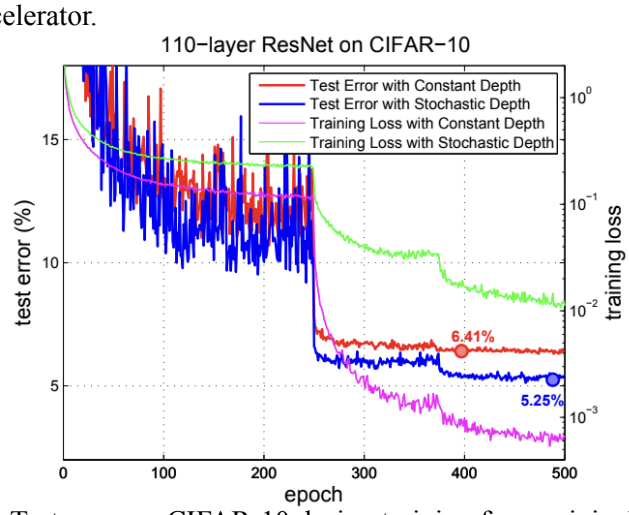

Figure 1. Test error on CIFAR-10 during training from original paper [1]

Table 1. Training time comparison of original paper on benchmark [1]

	CIFAR10+	CIFAR100+	SVHN
Constant Depth	20h 42m	20h 51m	33h 43m
Stochastic Depth	15h 7m	15h 20m	25h 33m

3. Methodology

In a normal ResNet, each residual block adds a learned residual transformation to an identity skip connection. Stochastic depth modifies this process by randomly dropping the residual block during training [1]:

$$H_\ell = ReLU(b_\ell f_\ell(H_\ell - 1) + id(H_\ell - 1)), \quad b_\ell \sim Bernoulli(p_\ell).$$

The survival probability follows a linear decay schedule [1]:

$$p_\ell = 1 - (\ell / L)(1 - p_L), \quad with\, p_L = 0.5.$$

Thus, deeper blocks are more likely to be skipped during training, while all blocks are used at test time.

In our experiment setup, both constant-depth and stochastic-depth models used the same ResNet-110 structure with 16, 32, and 64 filters across three stages. Each stage contains 18 residual blocks. Table 2 shows the basic training setup.

Table 2. Training setup for CIFAR-10 re-implementation of ResNet-110

Setting	Configuration
Dataset	CIFAR-10 (45k train / 5k val / 10k test, 32×32 color images, 10 class)
Data Augmentation	Horizontal flip + random 4px translation
Optimizer	SGD (momentum 0.9, weight decay 1e-4)
LR Schedule	lr=0.1(decay 10× at epochs 250, 375), 500 epochs total
Framework	PyTorch (original in Torch 7)
Batch Size / Device	128 / A100 GPU

We conducted four experiments to evaluate the behavior of stochastic depth.

- **Core reproduction**: compared final test error, training loss, and wall-clock training time between constant depth and stochastic depth.
- **Vanishing gradients monitor**: tracked the mean gradient magnitude of the first convolutional layer at each epoch during training.
- **Feature representation quality**: extracted penultimate-layer features and evaluated with t-SNE visualization, k-NN accuracy (k = 1, 5, 10, 20), silhouette score, and intra/inter-class ratio.
- **Label noise robustness**: corrupted 20%, 40%, 60% of training labels and tracked the training-testing error gap to compare how quickly each model overfits.

4. Results & Analysis

4.1 Core Reproduction

As shown in Figure 2 and Table 3, our results support the paper's central claims. Stochastic depth has higher training loss but better test performance, suggesting less overfitting and stronger generalization. Regarding training efficiency, stochastic depth achieved a 13.2% speedup compared to the constant-depth baseline, which is more modest than the original paper. We attribute this narrower gap to the improvement of modern GPU as both models in our setup train roughly eight times faster than reported in the original paper.

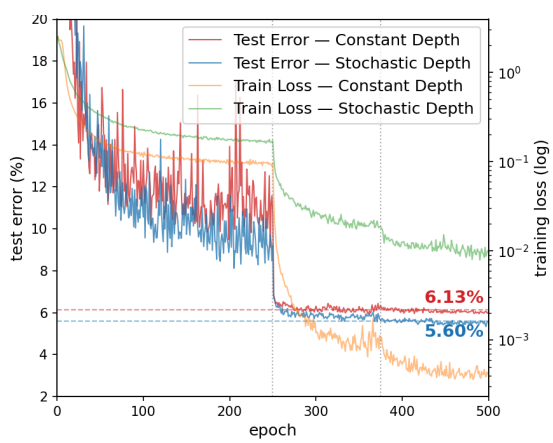

Figure 2. Test Error and Training Loss on CIFAR-10

Table 3. Wall-clock training time comparison

Method	Ours	Paper's (2016)
Constant depth	2h 51m	20h 42m
Stochastic depth	2h 28m	15h 7m
Wall-clock speedup	**13.2%**	~25%

4.2 Gradient Magnitude Analysis

After the learning rate drops, constant-depth gradients in the first convolutional layer collapse nearly to zero, whereas stochastic depth maintains meaningful gradient flow throughout. This confirms the paper's hypothesis that shorter effective training paths alleviate the vanishing gradient problem.

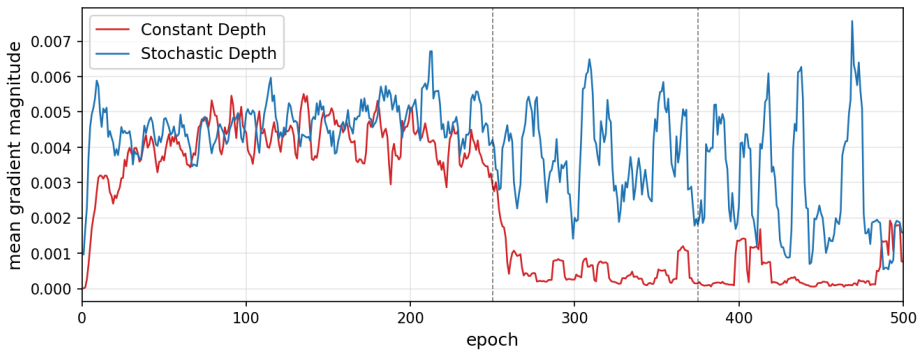

Figure 3. Stochastic Depth Alleviates Vanishing Gradients

4.3 Feature Representation Quality

Stochastic depth produces looser clusters as shown in Figure 4, and lower silhouette score as well as intra/inter distance ratio in Table 4, yet shows consistently higher k-NN accuracy. This apparent contradiction reveals that compact clusters reflect over-specialization, not richer representations. SD learns broader decision boundaries that generalize better to unseen data.

Table 4. 64-dimensional feature representation quality metrics

Metric	Constant Depth	Stochastic Depth
k-NN Accuracy (k=1)	91.20%	**91.40%**
k-NN Accuracy (k=5)	93.00%	**94.20%**
k-NN Accuracy (k=10)	93.00%	**94.60%**
k-NN Accuracy (k=20)	93.00%	**94.40%**
Silhouette Score (↑ better)	**0.6556**	0.5377
Intra/Inter Distance Ratio (↓ better)	**0.2524**	0.3898

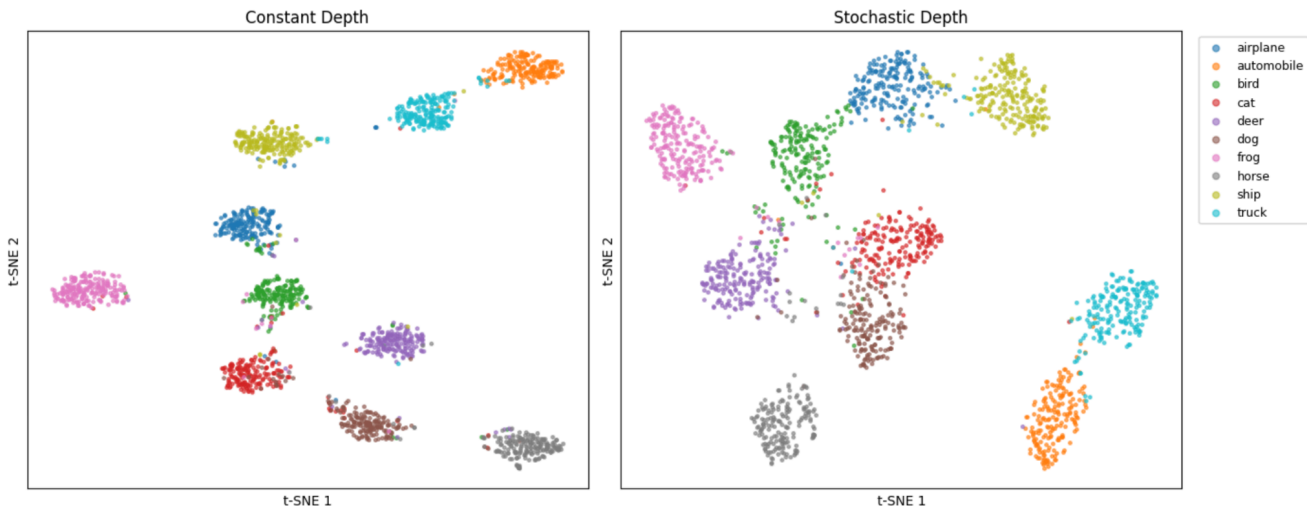

Figure 4. t-SNE of penultimate layer features

4.4 Label Noise Robustness

As observed in Figure 5, stochastic depth shows greater resistance to overfitting under corrupted training labels. At a noise level of 0.4, the constant-depth model drives its training error to near zero but plateaus at a test error of 20.08%, indicating severe overfitting to the noisy labels. Stochastic-depth mode, by contrast, maintains a notably higher training error but achieves a best test error at 15.4%, which is even lower than its own training error, suggesting that it resists memorizing noisy samples. This pattern holds across all noise levels, as shown in Figure 6, where stochastic depth consistently outperforms constant depth, with the gap widening from roughly 1% under clean labels to over 7% at a noise level of 0.6. This result supports the view that stochastic depth reduces overfitting and improves generalization as well as robustness.

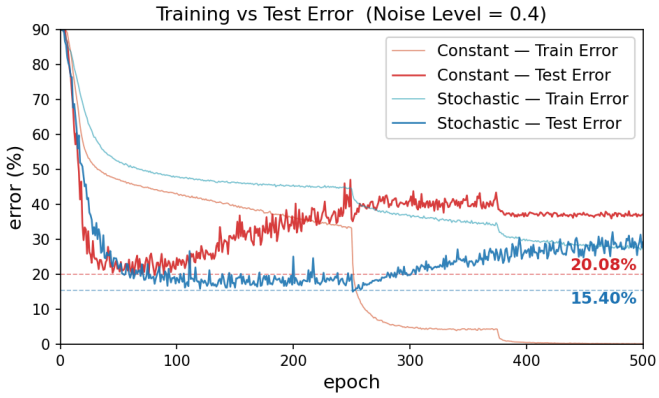

Figure 5. Testing and Training Error at Noise Level of 0.4

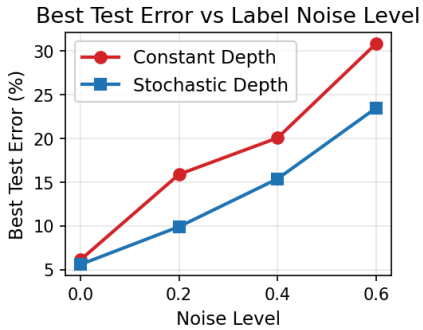

Figure 6. Best Test Error Across Different Noise Levels

5. Reflections

Reproducing this paper revealed that small implementation choices: training schedule, initialization, augmentation, and validation selection, noticeably affect final results. Our reproduction confirmed the main finding: stochastic depth improves test accuracy and reduces training time. And our additional analyses showed that stochastic depth also preserves stronger gradients, slows overfitting, and improves label-noise robustness. We also found that visually cleaner t-SNE representations do not always correlate with better generalization.

Future work would include multiple random seeds for statistical stability, evaluation on CIFAR-100 and SVHN, comparisons with alternative survival schedules and regularization methods such as Dropout and DropPath, and reproduction of the 1202-layer experiment.

6. References

[1] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, “Deep networks with stochastic depth,” *arXiv:1603.09382*, 2016.